



## Universidad de Cuenca

Facultad de Ingeniería  
Ingeniería en Telecomunicaciones

### Microprocesadores, Microcontroladores y Sistemas Embebidos

Kenneth S. Palacio Baus  
[kenneth.palacio@ucuenca.edu.ec](mailto:kenneth.palacio@ucuenca.edu.ec)

Jaime Patricio Chiqui Chiqui  
[jpatricio.chiquic@ucuenca.edu.ec](mailto:jpatricio.chiquic@ucuenca.edu.ec)

---

### Práctica 2

PIC18F4550 - Manejo de Temporizadores e Interrupciones

---

- **Valoración:** 10 puntos.
- **Tipo de Trabajo:** Trabajo práctico e informe individual.
- **Objetivos:** Mediante la presente práctica el estudiante aprenderá a:
  - Configurar el módulo temporizador TIMER de un microcontrolador PIC18F4550.
  - Reconocer la función del PRE-SCALER.
  - Reconocer los módulos temporizadores como una fuente de interrupción del microcontrolador.
  - Escribir una subrutina de atención a la interrupción generada por un timer.
- **Recursos:** Como base de esta práctica, utilizaremos MPLAB y la hoja de datos del microcontrolador PIC18F4550.

## Instrucciones

Para obtener una calificación en la presente práctica, cada estudiante deberá entregar un informe escrito según la estructura que se menciona más adelante. No olvide que puede contactar al profesor vía correo electrónico en caso que necesite asistencia adicional.

- Envíe su trabajo mediante la plataforma e-nova.
- El nombre del archivo de su informe debe tener el formato:  
*ApellidoNombre\_Pract02\_MicroCon\_M26.pdf*
- Si su envío no cumple con el nombre de archivo y fecha de entrega, no recibirá calificación.

# 1. Materiales y Equipos

Para el desarrollo de la presente práctica, se utilizaron los siguientes recursos y componentes electrónicos:

- **Software:** MPLAB IDE v8.92.
- **Microcontrolador:** PIC18F4550 de Microchip.
- **Resistencias:**
  - 3 de 10 k $\Omega$  (Pull-up para pulsantes y reset).
  - 9 de 1 k $\Omega$  (Limitadoras para los LEDs).
- **Oscilador:** Cristal de cuarzo de 20 MHz (XTAL).
- **Capacitores:** 2 cerámicos de 22 pF (para el cristal).
- **Pulsantes:** 3 micropulsantes (S1, S2, S3).
- **Visualización:** 9 diodos LED.
- **Programador:** *PICkit 3* con su respectivo cable USB.
- **Espadín:** Conector de 6 pines tipo espadín (peineta).
- **Otros:** Protoboard, cables de conexión calibre 22 (o jumpers).

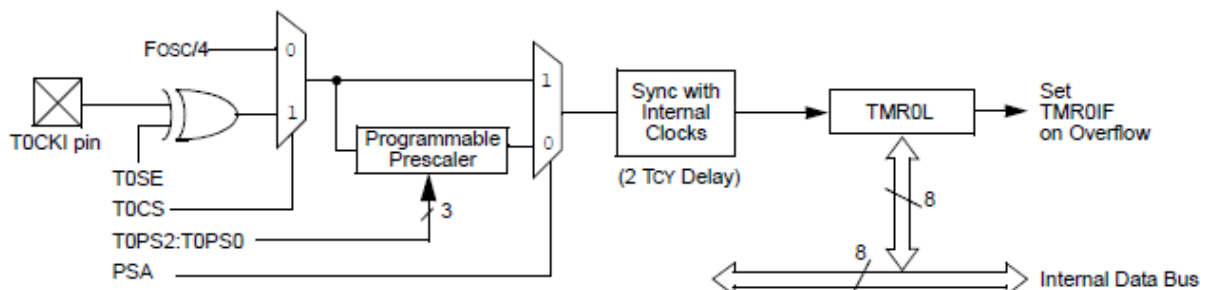
## 2. Marco Teórico

### 2.1. El Módulo Timer0 del PIC18F4550

El Timer0 es un módulo temporizador/contador de propósito general disponible en el PIC18F4550. Este módulo puede funcionar como temporizador (incrementándose con cada ciclo de instrucción interno) o como contador (incrementándose con pulsos externos). Para esta práctica, se utilizó en modo temporizador.

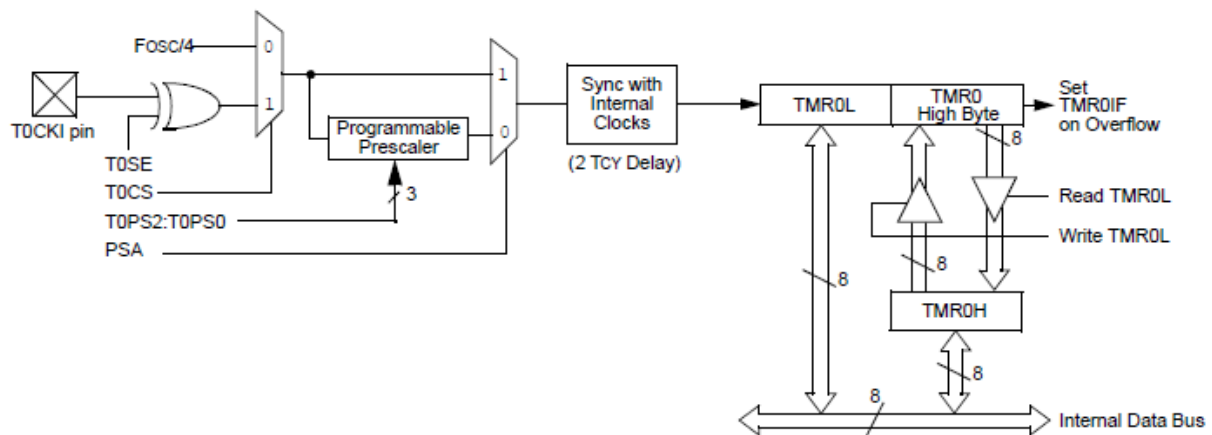
#### 2.1.1. Características Principales del Timer0

- **Resolución configurable:** Puede operar en modo de 8 bits o 16 bits.



**Note:** Upon Reset, Timer0 is enabled in 8-bit mode with clock input from T0CKI maximum prescale.

Figura 1: TMR0 - 8 Bits.



Note: Upon Reset, Timer0 is enabled in 8-bit mode with clock input from T0CKI maximum prescale.

Figura 2: TMR0 - 16 Bits.

- **Prescaler programmable:** Divisor de frecuencia configurable desde 1:2 hasta 1:256.
- **Fuente de reloj seleccionable:** Puede usar el reloj interno ( $F_{osc}/4$ ) o una señal externa.
- **Generación de interrupciones:** Al producirse un desbordamiento (overflow), se activa una bandera de interrupción.

### 2.1.2. Registros de Configuración del Timer0

El Timer0 se configura principalmente mediante el registro *T0CON* (Timer0 Control Register):

#### Registro T0CON (Timer0 Control Register) – PIC18F4550

Dirección: FD5h

Reset: 0000 0000b

| Bit | 7             | 6             | 5           | 4           | 3          | 2            | 1            | 0            |
|-----|---------------|---------------|-------------|-------------|------------|--------------|--------------|--------------|
|     | <b>TMR0ON</b> | <b>T08BIT</b> | <b>T0CS</b> | <b>T0SE</b> | <b>PSA</b> | <b>T0PS2</b> | <b>T0PS1</b> | <b>T0PS0</b> |
|     | R/W           | R/W           | R/W         | R/W         | R/W        | R/W          | R/W          | R/W          |

**TMR0ON**

Control de encendido del Timer0

1 = Timer0 encendido

0 = Timer0 apagado

**T08BIT**

Selección de 8 o 16 bits

1 = 8 bits (usa TMR0L)

0 = 16 bits (usa TMR0H y TMR0L)

**T0CS**

Selección de fuente de reloj

1 = Externo (pin T0CKI)

0 = Interno (Fosc/4)

**T0SE**

Selección de flanco del reloj externo (solo si T0CS=1)

1 = Flanco de bajada

0 = Flanco de subida

**PSA**

Asignación del prescaler

1 = Prescaler NO asignado al Timer0

0 = Prescaler asignado al Timer0

**T0PS2:T0PS0**

Selección de la relación del prescaler (solo si PSA = 0)

| T0PS2 | T0PS1 | T0PS0 | Relación |
|-------|-------|-------|----------|
| 0     | 0     | 0     | 1 : 2    |
| 0     | 0     | 1     | 1 : 4    |
| 0     | 1     | 0     | 1 : 8    |
| 0     | 1     | 1     | 1 : 16   |
| 1     | 0     | 0     | 1 : 32   |
| 1     | 0     | 1     | 1 : 64   |
| 1     | 1     | 0     | 1 : 128  |
| 1     | 1     | 1     | 1 : 256  |

Figura 3: Registro T0CON del Timer0

### Bits del Registro T0CON

- **TMR0ON (bit 7):** Habilita (1) o deshabilita (0) el Timer0.
- **T08BIT (bit 6):** Selecciona modo 8 bits (1) o 16 bits (0).
- **T0CS (bit 5):** Selecciona fuente: reloj interno (0) o externo (1).
- **T0SE (bit 4):** Selecciona flanco de reloj externo (si aplica).
- **PSA (bit 3):** Asigna prescaler: Timer0 (0) o WDT (1).
- **T0PS2:T0PS0 (bits 2-0):** Seleccionan el valor del prescaler.

## 2.2. Función del Prescaler

El prescaler es un divisor de frecuencia que permite reducir la velocidad a la que se incrementa el contador del Timer0. Esto es fundamental para lograr temporizaciones más largas sin necesidad de software adicional.

| T0PS2:T0PS0 | Valor del Prescaler | División        |
|-------------|---------------------|-----------------|
| 000         | 1:2                 | $F_{osc}/4/2$   |
| 001         | 1:4                 | $F_{osc}/4/4$   |
| 010         | 1:8                 | $F_{osc}/4/8$   |
| 011         | 1:16                | $F_{osc}/4/16$  |
| 100         | 1:32                | $F_{osc}/4/32$  |
| 101         | 1:64                | $F_{osc}/4/64$  |
| 110         | 1:128               | $F_{osc}/4/128$ |
| 111         | 1:256               | $F_{osc}/4/256$ |

Cuadro 1: Valores del Prescaler del Timer0

### Ejemplo de cálculo:

Con un cristal de 20 MHz:

- Frecuencia de instrucción:  $F_{cy} = \frac{20 \text{ MHz}}{4} = 5 \text{ MHz}$
- Período de instrucción:  $T_{cy} = \frac{1}{5 \text{ MHz}} = 0,2\mu s$
- Con prescaler 1:256:  $T_{prescaler} = 0,2\mu s \times 256 = 51,2\mu s$

## 2.3. Interrupciones en Microcontroladores

Una interrupción es un mecanismo que permite al microcontrolador detener temporalmente la ejecución del programa principal para atender un evento específico (como el desbordamiento de un temporizador). Una vez atendida la interrupción, el programa principal se reanuda desde donde fue interrumpido.

### 2.3.1. Vectores de Interrupción

El PIC18F4550 tiene dos vectores de interrupción:

- **Vector de Alta Prioridad (0x0008):** Para interrupciones urgentes.
- **Vector de Baja Prioridad (0x0018):** Para interrupciones menos críticas.

En esta práctica se utilizó el vector de alta prioridad:

```
1 org    0x08        ; Vector de interrupcion HP
2 goto   InterrupcionHP
```

### 2.3.2. Registro INTCON

El registro INTCON controla las interrupciones globales y específicas:

- **GIE (bit 7):** Habilita todas las interrupciones globalmente.
- **PEIE (bit 6):** Habilita interrupciones de periféricos.
- **TMR0IE (bit 5):** Habilita la interrupción del Timer0.
- **TMR0IF (bit 2):** Bandera que indica desbordamiento del Timer0.

#### Importante: Limpieza de Banderas

La bandera TMR0IF debe ser limpiada manualmente dentro de la rutina de servicio de interrupción (ISR) mediante la instrucción:

BCF INTCON, TMR0IF

Si no se limpia, la interrupción se volverá a ejecutar inmediatamente al salir de la ISR.

| Name    | Bit 7                     | Bit 6                 | Bit 5   | Bit 4   | Bit 3  | Bit 2  | Bit 1  | Bit 0  | Reset Values on page |
|---------|---------------------------|-----------------------|---------|---------|--------|--------|--------|--------|----------------------|
| TMR0L   | Timer0 Register Low Byte  |                       |         |         |        |        |        |        | 54                   |
| TMR0H   | Timer0 Register High Byte |                       |         |         |        |        |        |        | 54                   |
| INTCON  | GIE/GIEH                  | PEIE/GIEL             | TMR0IE  | INT0IE  | RBIE   | TMR0IF | INT0IF | RBIF   | 53                   |
| INTCON2 | RBPV                      | INTEDG0               | INTEDG1 | INTEDG2 | —      | TMR0IP | —      | RBIP   | 53                   |
| T0CON   | TMR0ON                    | T08BIT                | T0CS    | T0SE    | PSA    | T0PS2  | T0PS1  | T0PS0  | 54                   |
| TRISA   | —                         | TRISA6 <sup>(1)</sup> | TRISA5  | TRISA4  | TRISA3 | TRISA2 | TRISA1 | TRISA0 | 56                   |

Figura 4: REGISTERS ASSOCIATED WITH TIMER0

Un diagrama de bloque de resumen general se aprecia en la siguiente Figura:

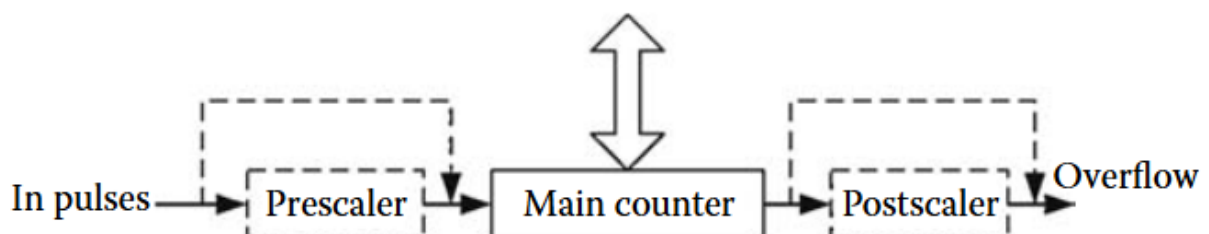


Figura 5: General block diagram for medium-end PIC timers. Obtenido de [3]

### 3. Diseño del Sistema Microcontrolado

#### 3.1. Plataforma de Hardware

Se implementó el circuito de la Figura 6 en un protoboard, reutilizando el mismo hardware de la Práctica 1.

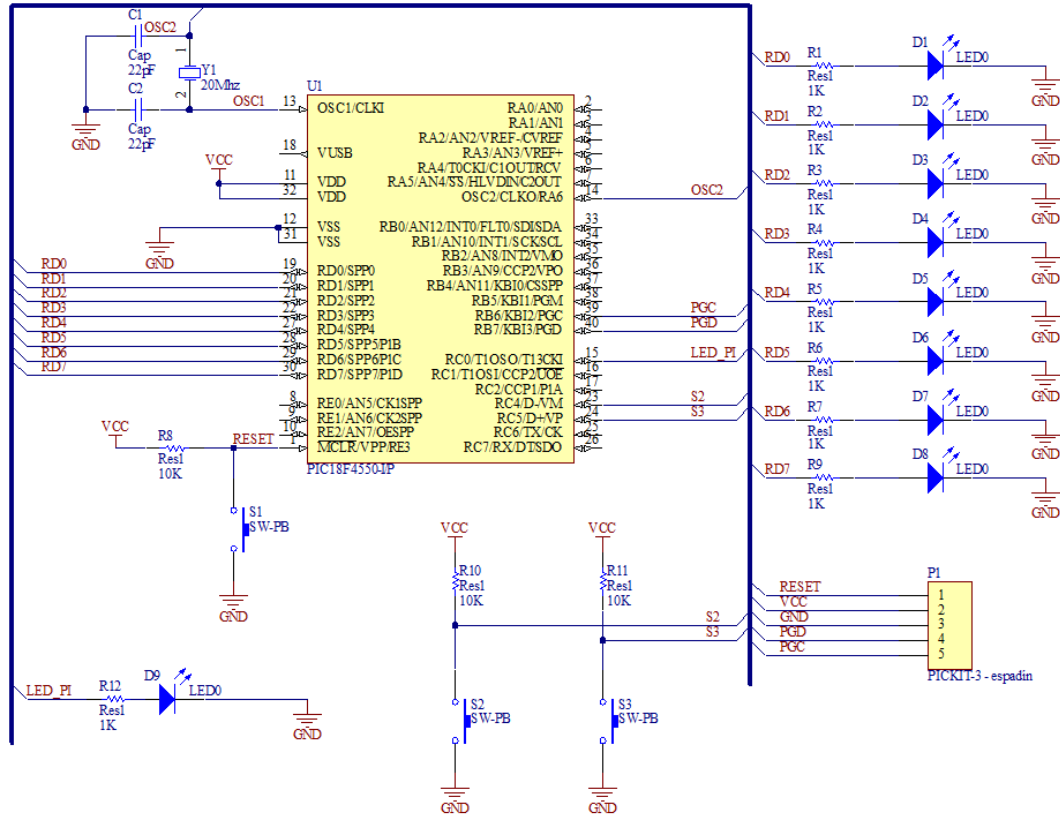


Figura 6: Circuito para manejo de Temporizador e Interrupciones: PIC18F4550

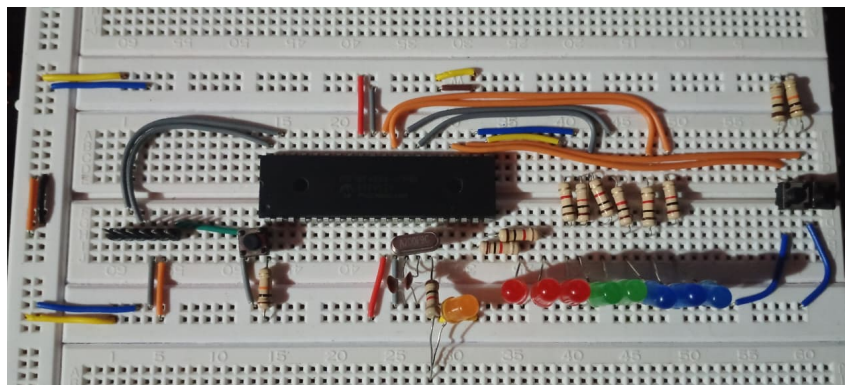


Figura 7: Implementación física del circuito en protoboard

Los componentes cumplen las mismas funciones descritas en la Práctica 1, con la diferencia de que ahora el LED piloto (RC0) es controlado por el Timer0 mediante interrupciones.

## 3.2. Funcionamiento del Software

### 3.2.1. Secuencia de Inicio

*Implemente una secuencia de inicio en la que el LED piloto en RC0 parpadee con un tiempo de encendido y apagado de 50ms. Luego 20 parpadeos, este LED debe quedar encendido.*

Al igual que en la práctica anterior, se implementa una secuencia de inicio donde el LED piloto parpadea 20 veces:

```
1 ;----- SECUENCIA DE INICIO -----
2 MOVLW    d'20'
3 MOVWF    ContadorInicio ;RC0 parpadea 20 veces / 50 ms
4
5 BucleInicio:
6     BSF    LATC, 0          ; Enciende LED RC0-Led_piloto
7     MOVLW  d'50'
8     CALL   DelayXms
9     BCF    LATC, 0          ; Apaga LED RC0-Led_piloto
10    MOVLW  d'50'
11    CALL   DelayXms
12    DECFSZ ContadorInicio, 1
13    GOTO    BucleInicio
14
15    BSF    LATC, 0          ; RC0 ENCENDIDO
```

Listing 1: Secuencia de inicio

### 3.3. PARTE 1: Secuencia Ola con Pausa y Continuación

#### 3.3.1. Descripción de la Funcionalidad

En esta parte se implementó una secuencia tipo "ola" donde dos LEDs permanecen siempre encendidos y los demás se desplazan de izquierda a derecha y viceversa. Las características principales son:

- La secuencia se inicia al presionar S2
- Se puede pausar en cualquier momento con S3 (sin apagar los LEDs)
- Al presionar S2 nuevamente, continúa desde donde se pausó
- El tiempo entre transiciones es de 100 ms

#### 3.3.2. Código Completo - Parte 1

```
1 ; PRACTICA 2 - MICROCONTROLADORES
2 ; PARTE 1: Secuencia Ola con leds y pausa
3 ; MARZO-AGOSTO 2026
4 ; Jaime Chiqui
5
6 ;*****
7 ; Program Header
8     LIST p=18f4550
9     #include "p18f4550.inc"
10
11 ;***** DECLARACION VARIABLES *****
12 Contador1      equ 0x00
13 Contador2      equ 0x01
14 ContaDelay100  equ 0x02
15 Xveces         equ 0x03
16 ContadorInicio equ 0x04
17 Sentido        equ 0x05
18
19 ;----- Definicion de Vectores -----
20     org      0x00
21     goto     Inicio
22
23 ; ++++ INICIO DEL PROGRAMA ++++++
24     org      0x20
25 Inicio:
26
27 ;-----
28 ; Configuracion de perifericos
29     CLRF     TRISD      ; TRISD = 00000000
30     BCF      UCON, 3    ; RC4 Y RC5 entradas
31     BSF      UCFG, 3    ; Especial RC4 Y RC5
32     BCF      TRISC,0    ; led_piloto
33
34 ;***** Inicializacion de I/O *****
35     CLRF     LATD
36     BCF      LATC, 0
37
38 ;--- INICIALIZACION DE VARIABLES -----
39     MOVLW    d'198'
40     MOVWF    Contador1
41     MOVLW    d'21'
42     MOVWF    Contador2
43     MOVLW    d'20'
44     MOVWF    ContadorInicio
```



```

45     MOVLW    d'100'
46     MOVWF    ContaDelay100
47
48 ;----- SECUENCIA DE INICIO -----
49 BucleInicio:
50     BSF      LATC, 0
51     MOVLW    d'50'
52     CALL     DelayXms
53     BCF      LATC, 0
54     MOVLW    d'50'
55     CALL     DelayXms
56     DECFSZ   ContadorInicio, 1
57     GOTO     BucleInicio
58
59     BSF      LATC, 0
60
61 ;-----
62 EncuestaON:
63     BTFSC    PORTC, 4
64     GOTO     EncuestaON
65     CALL     Delay100ms
66 Est_transistorio:
67     BTFSS    PORTC, 4
68     GOTO     Est_transistorio
69
70 ;***** SECUENCIA OLA *****
71     MOVLW    b'00000011' ; 2 LEDs siempre encendidos
72     MOVWF    LATD
73
74 Ola_Hacia_Izq:
75     MOVLW    d'100'
76     CALL     DelayXms
77
78     BTFSS    PORTC, 5 ; verificar boton s3
79     GOTO     Detener_Secuencia
80     BCF      Sentido, 0 ; bandera sent. izq.
81
82     RLNCF    LATD, 1
83     BTFSS    LATD, 7
84     GOTO     Ola_Hacia_Izq
85
86 Ola_Hacia_Der:
87     MOVLW    d'100'
88     CALL     DelayXms
89
90     BTFSS    PORTC, 5
91     GOTO     Detener_Secuencia
92     BSF      Sentido, 0 ; bandera sent. DER.
93
94     RRNCF    LATD, 1
95     BTFSS    LATD, 0
96     GOTO     Ola_Hacia_Der
97
98     GOTO     Ola_Hacia_Izq
99
100 ; ----- DETENER Y NO OFF
101 Detener_Secuencia:
102     CALL     Delay100ms
103 Boton_S3_D:
104     BTFSS    PORTC, 5
105     GOTO     Boton_S3_D
106
107 ; ----- CONTINUA S2

```

```

108 Trans_s2:
109     BTFSC    PORTC, 4
110     GOTO     Trans_s2
111     CALL     Delay100ms
112 Continuar_s2:
113     BTFSS    PORTC, 4
114     GOTO     Continuar_s2
115
116     BTFSC    Sentido, 0      ; direccion?
117     GOTO     Ola_Hacia_Der
118     GOTO     Ola_Hacia_Izq
119
120 ;++++++ DELAY DE X[ms] ++++++
121 DelayXms:
122     MOVWF    Xveces
123 bucleVeces:
124     CALL     Delay1ms
125     DECFSZ   Xveces,1
126     GOTO     bucleVeces
127     return
128
129 ;++++++ DELAY DE 100MS ++++++
130 Delay100ms:
131     CALL     Delay1ms
132     DECFSZ   ContaDelay100,1
133     GOTO     Delay100ms
134     MOVLW    d'100'
135     MOVWF    ContaDelay100
136     return
137
138 ;++++++ DELAY DE 1MS ++++++
139 Delay1ms:
140     DECFSZ   Contador2,1
141     GOTO     bucle1
142     GOTO     salir
143 bucle1:
144     DECFSZ   Contador1,1
145     GOTO     bucle1
146
147     MOVLW    d'198'
148     MOVWF    Contador1
149     GOTO     Delay1ms
150 salir:
151     MOVLW    d'21'
152     MOVWF    Contador2
153     return
154
155 end

```

Listing 2: Código Parte 1 - Secuencia Ola con Pausa

### 3.3.3. Explicación del Funcionamiento - Parte 1

#### 1. Variable de Sentido:

Se utiliza la variable **Sentido** para recordar en qué dirección se movía la secuencia cuando se pausó:

- **Sentido = 0:** La ola se movía hacia la izquierda
- **Sentido = 1:** La ola se movía hacia la derecha

#### 2. Detección de Pausa (S3):

Durante la ejecución de la secuencia, se verifica constantemente el estado de S3:

```

1 BTFSS   PORTC, 5      ; S3
2 GOTO    Detener_Secuencia

```

Si S3 está presionado (nivel bajo), el programa salta a **Detener\_Secuencia**, donde espera a que se suelte el botón sin modificar el estado de los LEDs.

### 3. Continuación desde la Pausa:

Cuando se presiona S2 nuevamente, el programa evalúa la variable **Sentido** para determinar a qué bucle debe saltar:

```

1 BTFSC   Sentido, 0      ; Esta en dir. derecha?
2 GOTO    Ola_Hacia_Der   ; Si, va a derecha
3 GOTO    Ola_Hacia_Izq   ; No, va a izquierda

```

### 4. Secuencia Ola:

La secuencia implementada utiliza rotaciones circulares (RLNCF y RRNCF) para desplazar los bits encendidos. El patrón inicial es **b'00000011'**, manteniendo siempre dos LEDs encendidos. Se observa la transición de colores al pasar del bit RD3 (Azul) al RD4 (Rojo) y el ciclo de retorno.

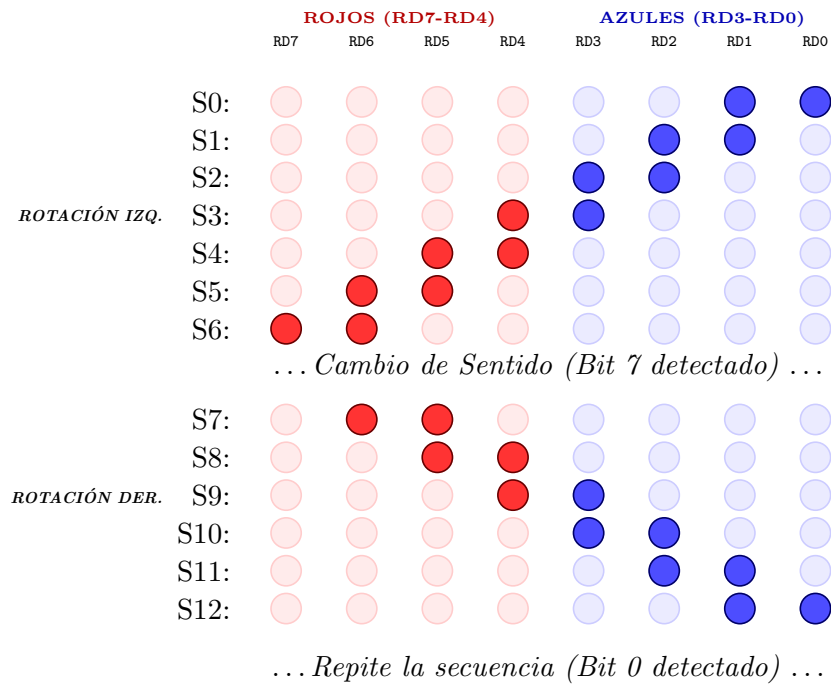


Figura 8: Secuencia Ola implementada.

### 3.4. PARTE 2: Timer0 Controlando el LED Piloto

#### 3.4.1. Descripción de la Funcionalidad

En esta parte se añade la funcionalidad del Timer0 para controlar el LED piloto (RC0) mediante interrupciones, haciendo que parpadee a 1 Hz (500 ms ON, 500 ms OFF) de forma independiente de la secuencia principal.

#### 3.4.2. Código Completo - Parte 2

```
1  ; PRACTICA 2 - MICROCONTROLADORES
2  ; PARTE 2: Timer0 en led Piloto (500 ms)
3  ; MARZO-AGOSTO 2026
4  ; Jaime Chiqui
5
6  ;***** ; Program Header
7  LIST p=18f4550 ;Especifica el dispositivo
8  #include "p18f4550.inc" ;traducción de variables a memoria
9  ;*****
10 ;***** DECLARACION VARIABLES *****
11 Contador1 equ 0x00
12 Contador2 equ 0x01
13 ContaDelay100 equ 0x02
14 Xveces equ 0x03
15 ContadorInicio equ 0x04
16 Sentido equ 0x05
17 ;*****
18
19 ;----- Definición de Vectores -----
20 org 0x00 ; Vector de Inicio / reset
21 goto Inicio
22 ;-----
23 org 0x08 ; Vect. interr. HP
24 goto InterrupcionHP
25 ; ++++ INICIO DEL PROGRAMA ++++++
26 org 0x20 ; Dnde comienza el programa
27 Inicio:
28
29 ;#####
30 ; Configuración de periféricos
31 ; -----
32 ; CONFIGURACION DE ENTRADA/SALIDA
33 ; .....
34 BCF TRISC,0 ; Apaga LED piloto
35 CLRF TRISD ; Puerto D como salida
36 BCF TRISC, 0 ; RC0 salida (LED piloto)
37 CLRF LATD ; Apaga LEDs
38 BCF UCON,3 ;RC4 Y RC5 ENTRADAS
39 BSF UCFG,3 ;ESPECIAL DE RC4-RC5
40 ; -----
41 ; CONFIGURACION DEL TIMER0
42 ; .....
43 ; Configurar bit a bit (no recomendable)
44 BCF TOCON, TMROON ; Apaga TMRO (USA TMROON 0 7: TOCON, 7)
45 BCF TOCON, T08BIT ; TMRO A 16 Bits
46 BCF TOCON, TOCS ; TMRO - MODO TIMER (no pull up)
47
48 BCF TOCON, PSA ; Prescaler SI
49 BSF TOCON, TOPS2 ; Prescaler 256
50 BSF TOCON, TOPS1 ;
51 BSF TOCON, TOPS0 ;
```

```

52
53 ; -----
54 ; CONFIGURACION DE INTERRUPCIONES
55 ; .....
56 BSF      INTCON, GIE      ; Enceneder todas las interrupciones
57 BSF      INTCON, PEIE     ; Prendo Interr. PERIFERICOS
58 BSF      INTCON, TMROIE   ; Prendi Interr. T0
59
60 ;--- INICIALIZACION DE VARIABLES -----
61 MOVLW    d'198'           ;W = 198
62 MOVWF    Contador1        ;Contador1 = 10
63 MOVLW    d'21'           ;W = 21
64 MOVWF    Contador2        ;Contador2 = 10
65 MOVLW    d'20'           ;
66 MOVWF    ContadorInicio   ;RC0 parpadea 20 veces / 50 ms
67 MOVLW    d'100'          ;
68 MOVWF    ContaDelay100    ;
69
70 MOVLW    0xD9             ; 65535 (55769) ---> CAMBIA HEXADECIMAL
71 MOVWF    TMR0H            ; PARTRE ALTA 16 BITS
72 MOVLW    0xDA             ; AL CAMBIAR CF A D5 ESTE CUENTA MENOS YA SI
73 MOVWF    TMR0L            ; PARTRE BAJA 16 BITS
74
75 BSF      TOCON, TMROON    ; Enciende el Timer0
76
77 ;----- SECUENCIA DE INICIO -----
78 BucleInicio:
79 BSF      LATC, 0           ; Enciende LED RC0-Led_piloto
80 MOVLW    d'50'            ;
81 CALL     DelayXms          ;
82 BCF      LATC, 0           ; Apaga LED RC0-Led_piloto
83 MOVLW    d'50'            ;
84 CALL     DelayXms          ;
85 DECFSZ   ContadorInicio, 1 ; Decrementa y salta si es cero
86 GOTO     BucleInicio
87
88 BSF      LATC, 0           ; RC0-Led_piloto ENCENDIDO
89 ;-----
90 EncuestaON:                ; Espera a S2 para iniciar secuencia
91 BTFSC    PORTC, 4
92 GOTO     EncuestaON
93 CALL     Delay100ms
94 Est_transistorio:          ; Esperar a que suelte S2
95 BTFSS    PORTC, 4
96 GOTO     Est_transistorio
97
98 ;***** SECUENCIA OLA ***** ; Inicio de la secuencia
99 solicitada
100 MOVLW    b'00000011'      ; LED izq ON (rojos) siempre encendido
101 MOVWF    LATD
102
103 Ola_Hacia_Izq:
104 MOVLW    d'100'           ; Carga 100ms para el delay
105 CALL     DelayXms
106
107 BTFSS    PORTC, 5          ; verificar boton s3
108 GOTO     Detener_Secuencia
109 BCF      Sentido, 0        ; bandera indicar sent. izq.
110
111 RLNCF    LATD, 1           ; Rota la luz a la izquierda
112 BTFSS    LATD, 7           ; 10000000 (bit 7 en 1)
113 GOTO     Ola_Hacia_Izq

```

```

114 Ola_Hacia_Der:
115     MOVLW    d'100'           ; Carga 100ms para el delay
116     CALL     DelayXms
117
118     BTFSS    PORTC, 5         ; verificar boton s3
119     GOTO     Detener_Secuencia
120     BSF      Sentido, 0       ; bandera indicar sent. DER.
121
122     RRNCF    LATD, 1          ; Rota la luz a la derecha
123     BTFSS    LATD, 0          ; 00000001 (bit 0 es 1)
124     GOTO     Ola_Hacia_Der
125
126     GOTO     Ola_Hacia_Izq    ; Repite la Ola
127 ; ----- DETENER Y NO OFF
128     Detener_Secuencia:
129     CALL     Delay100ms       ; Anti-rebote
130     Boton_S3_D:
131     BTFSS    PORTC, 5         ; Espera que se suelte S3
132     GOTO     Boton_S3_D
133
134
135 ; ----- CONTINUA S2
136     Trans_s2:
137     BTFSC    PORTC, 4
138     GOTO     Trans_s2
139     CALL     Delay100ms       ; Anti-rebote
140     Continuar_s2:
141     BTFSS    PORTC, 4
142     GOTO     Continuar_s2
143
144     BTFSC    Sentido, 0       ; Esta en dir. derecha
145     GOTO     Ola_Hacia_Der    ; va a derecha
146     GOTO     Ola_Hacia_Izq    ; No, va a izquierda
147
148 ;++++++ DELAY DE X[ms] ++++++
149 ;Deberia llamar a Delay1ms, Xveces
150 DelayXms:
151     MOVWF    Xveces
152     bucleVeces:
153     CALL     Delay1ms
154     DECFSZ   Xveces,1
155     GOTO     bucleVeces
156
157     return
158 ;+++++
159
160 ;++++++ DELAY DE 100MS ++++++
161 Delay100ms:
162     CALL     Delay1ms
163     DECFSZ   ContaDelay100,1
164     GOTO     Delay100ms
165     ;Reinicio ContaDelay100 = 100
166     MOVLW    d'100'           ;W=100
167     MOVWF    ContaDelay100
168
169     return
170 ;+++++
171 ;++++++ DELAY DE 1MS ++++++
172 Delay1ms:
173     DECFSZ   Contador2,1
174     GOTO     bucle1
175     GOTO     salir
176 bucle1:

```

```

177      DECFSZ   Contador1,1
178      GOTO     bucle1
179
180      MOVLW    d'198'      ;W = 198
181      MOVWF    Contador1   ;Contador1 = 198
182      GOTO     Delay1ms
183  salir:
184      MOVLW    d'21'      ;W = 21
185      MOVWF    Contador2   ;Contador2 = 10
186      return
187      ;+++++
188
189      ; *****INTERRUPCION_HP*****
190  InterrupcionHP:
191      ; Complementa LED RCO (D9)
192      BTG      LATC, 0      ; Toggle LED piloto
193      BCF      INTCON, TMR0IF ; OFF bandera IT TIMER
194
195      MOVLW    0xD9        ; 65536-9766 (55769=D9DA) ---> CAMBIA HEXADECIMAL
196      MOVWF    TMR0H        ; PARTRE ALTA 16 BITS
197      MOVLW    0xDA        ; AL CAMBIAR CF A D5 ESTE CUENTA MENOS Y ASI
198      MOVWF    TMR0L        ; PARTRE BAJA 16 BITS
199
200      RETFIE          ; Regresa de interrupci n
201      ;+++++
202  end
203      ;+++++

```

Listing 3: Código Parte 2 - Timer0 en led Piloto (500 ms)

### 3.4.3. Configuración del Timer0

```

1  ; -----
2  ; CONFIGURACION DEL TIMER0
3  ; -----
4  BCF      TOCON, TMR0ON    ; Apaga TMR0
5  BCF      TOCON, T08BIT    ; TMR0 a 16 bits
6  BCF      TOCON, TOCS      ; Modo TIMER (reloj interno)
7
8  BCF      TOCON, PSA       ; Prescaler SI
9  BSF      TOCON, TOPS2     ; Prescaler 256
10 BSF      TOCON, TOPS1
11 BSF      TOCON, TOPS0
12
13 ; -----
14 ; CONFIGURACION DE INTERRUPCIONES
15 ; -----
16 BSF      INTCON, GIE      ; Interrupciones globales
17 BSF      INTCON, PEIE     ; Interrupciones perifericos
18 BSF      INTCON, TMR0IE   ; Interrupcion Timer0
19
20 ;--- Valor para tener 500ms en el TMR0 ---
21 MOVLW    0xD9
22 MOVWF    TMR0H            ; Parte alta
23 MOVLW    0xDA
24 MOVWF    TMR0L            ; Parte baja
25
26 BSF      TOCON, TMR0ON    ; Enciende el Timer0

```

Listing 4: Configuración del Timer0

Explicación bit por bit:

- BCF TOCON, T08BIT: Configura el Timer0 en modo 16 bits, permitiendo contar hasta 65535.
- BCF TOCON, T0CS: Selecciona el reloj interno ( $F_{osc}/4 = 5 \text{ MHz}$ ).
- BCF TOCON, PSA: Asigna el prescaler al Timer0.
- BSF TOCON, T0PS2-T0PS0: Configura prescaler en 1:256 (111 binario).

#### 3.4.4. Rutina de Servicio de Interrupción (ISR)

```

1 ;----- Definicion de Vectores -----
2     org      0x00
3     goto     Inicio
4 ;-----
5     org      0x08      ; Vector interrupcion HP
6     goto     InterrupcionHP
7
8 .....
9
10 ; *****INTERRUPCION_HP*****
11 InterrupcionHP:
12     BTG      LATC, 0          ; Toggle LED piloto
13     BCF      INTCON, TMR0IF    ; Limpia bandera
14
15     MOVLW    0xD9
16     MOVWF    TMR0H
17     MOVLW    0xDA
18     MOVWF    TMR0L            ; Recarga valores
19
20     RETFIE                    ; Retorna de interrupcion

```

Listing 5: Rutina de interrupción del Timer0

#### Explicación:

- BTG LATC, 0: Complementa el estado del LED piloto (si estaba encendido se apaga y vice-versa).
- BCF INTCON, TMR0IF: Limpia la bandera de interrupción para evitar re-entradas.
- Se recargan los valores de TMR0H y TMR0L para el siguiente ciclo de 500 ms.
- RETFIE: Retorna de la interrupción, restaurando el contexto del programa principal.

#### Importante

La rutina de interrupción debe ser lo más corta posible para no retrasar el programa principal. En este caso, solo se complementa el LED y se recargan los registros del timer.

### 3.5. Cálculo de Temporización para 500 ms

Para generar una interrupción cada 500 ms con el Timer0 en modo 16 bits, se realizan los siguientes cálculos:

#### Datos:

- Frecuencia del cristal:  $F_{osc} = 20 \text{ MHz}$
- Prescaler seleccionado: 1:256
- Tiempo deseado:  $T_{deseado} = 500 \text{ ms} = 0.5 \text{ s}$



- Frecuencia de instrucción:  $F_{cy} = \frac{20 \text{ MHz}}{4} = 5 \text{ MHz}$
- Período de instrucción:  $T_{cy} = \frac{1}{5 \text{ MHz}} = 0,2\mu s$

**Paso 1: Calcular el período con prescaler**

$$T_{tick} = T_{cy} \times \text{Prescaler} = 0,2\mu s \times 256 = 51,2\mu s$$

**Paso 2: Calcular cuentas necesarias**

$$\#\_Cuentas = \frac{T_{deseado}}{T_{tick}} = \frac{0,5s}{51,2\mu s} = 9765,625 \approx 9766 \text{ cuentas}$$

**Paso 3: Calcular valor de precarga**

En modo 16 bits, el Timer0 cuenta de 0 a  $2^{16} - 1 = 65535$ . Para que desborde después de 9766 cuentas:

$$\text{Valor}_{precarga} = 65536 - 9766 = 55770$$

**Paso 4: Convertir a hexadecimal**

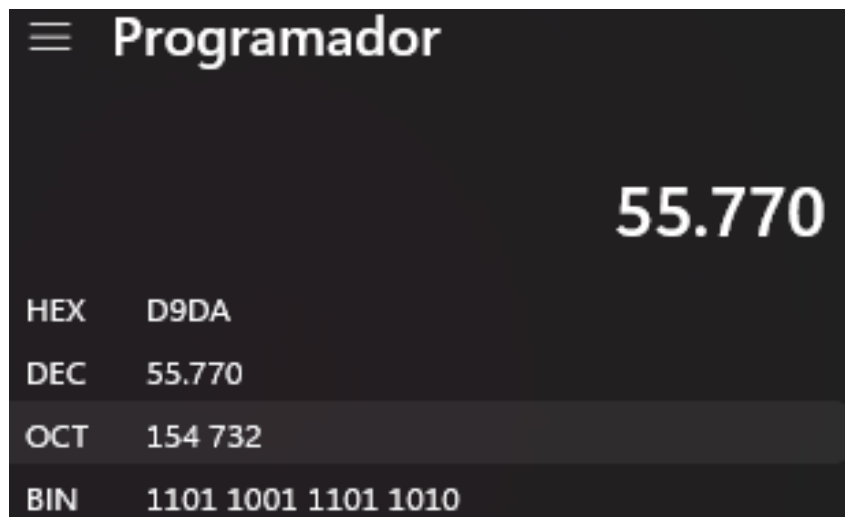


Figura 9: Conversiones

$$55770_{10} = D9DA_{16}$$

Por lo tanto:

- TMR0H = 0xD9
- TMR0L = 0xDA

**Verificación:**

$$T_{real} = 9766 \times 51,2\mu s = 500,0192ms \approx 500ms$$

El error es menor al 0.004 %, totalmente aceptable para esta aplicación.

## 3.6. PARTE 3: Temporizador de 15 Segundos

### 3.6.1. Descripción de la Funcionalidad

En esta parte se añade un contador de tiempo que limita la duración total de la secuencia a exactamente 15 segundos desde que se presiona S2 por primera vez, independientemente de las pausas.

### 3.6.2. Código Completo - Parte 3

```
1  ; PRACTICA 2 - MICROCONTROLADORES
2  ; PARTE 3: Secuencia Ola con Temporizador de 15 Segundos
3  ; MARZO-AGOSTO 2026
4  ; Jaime Chiqui
5
6  ;***** ; Program Header
7  LIST p=18f4550 ;Especifica el dispositivo
8  #include "p18f4550.inc" ;traducción de variables a memoria
9  ;*****
10 ;***** DECLARACION VARIABLES *****
11 Contador1 equ 0x00
12 Contador2 equ 0x01
13 ContaDelay100 equ 0x02
14 Xveces equ 0x03 ; Timers manuales
15 ContadorInicio equ 0x04 ; Cuenta 20 veces ON/OFF led piloto
16 Sentido equ 0x05 ; Sentido izq. o der. de la ola de leds
17 Contador15s equ 0x06 ; Cuenta 30 interrupciones (15 seg)
18 Flag equ 0x07 ; 0=Conteo ON, 1=OFF(Ya esta los 15 seg)
19 ;*****
20
21 ;----- Definición de Vectores -----
22 org 0x00 ; Vector de Inicio / reset
23 goto Inicio
24 ;-----
25 org 0x08 ; Vect. interr. HP
26 goto InterruccionHP
27
28 ; ++++ INICIO DEL PROGRAMA ++++++
29 org 0x20 ; Dnde comienza el programa
30 Inicio:
31
32 ;#####
33 ; Configuración de periféricos
34 ; -----
35 ; CONFIGURACION DE ENTRADA/SALIDA
36 ;.....
37 BCF TRISC,0 ; Apaga LED piloto
38 CLRF TRISD ; Puerto D como salida
39 BCF TRISC, 0 ; RC0 salida (LED piloto)
40 CLRF LATD ; Apaga LEDs
41 BCF UCON,3 ;RC4 Y RC5 ENTRADAS
42 BSF UCFG,3 ;ESPECIAL DE RC4-RC5
43 ; -----
44 ; CONFIGURACION DEL TIMERO
45 ;.....
46 ; Configurar bit a bit (no recomendable)
47 BCF TOCON, TMROON ; Apaga TMRO (USA TMROON 0 7: TOCON, 7)
48 BCF TOCON, T08BIT ; TMRO A 16 Bits
49 BCF TOCON, TOCS ; TMRO - MODO TIMER (no pull up)
50
51 BCF TOCON, PSA ; Prescaler SI
```

```

52      BSF      TOCON, TOPS2      ; Prescaler 256
53      BSF      TOCON, TOPS1      ;
54      BSF      TOCON, TOPS0      ;
55
56      ; -----
57      ; CONFIGURACION DE INTERRUPTACIONES
58      ; .....
59      BSF      INTCON, GIE      ; Encender todas las interrupciones
60      BSF      INTCON, PEIE      ; Prendo Interr. PERIFERICOS
61      BSF      INTCON, TMROIE      ; Prendi Interr. T0
62
63      ;--- INICIALIZACION DE VARIABLES -----
64      MOVLW     d'198'          ; W = 198
65      MOVWF     Contador1      ; Contador1 = 10
66      MOVLW     d'21'          ; W = 21
67      MOVWF     Contador2      ; Contador2 = 10
68      MOVLW     d'20'          ;
69      MOVWF     ContadorInicio ; RCO parpadea 20 veces / 50 ms
70      MOVLW     d'100'         ;
71      MOVWF     ContaDelay100 ;
72
73      ;--- Valores del TMRO ---
74      MOVLW     0xD9          ; 65535 (55769) ---> CAMBIA HEXADECIMAL
75      MOVWF     TMROH          ; PARTRE ALTA 16 BITS
76      MOVLW     0xDA          ; AL CAMBIAR CF A D5 ESTE CUENTA MENOS YA SI
77      MOVWF     TMROL          ; PARTRE BAJA 16 BITS
78
79      BSF      TOCON, TMROON      ; Enciende el TMRO
80
81      ;----- SECUENCIA DE INICIO -----
82      BucleInicio:
83      BSF      LATC, 0          ; Enciende LED RCO-Led_piloto
84      MOVLW     d'50'          ;
85      CALL     DelayXms
86      BCF      LATC, 0          ; Apaga LED RCO-Led_piloto
87      MOVLW     d'50'          ;
88      CALL     DelayXms
89      DECFSZ    ContadorInicio, 1 ; Decrementa y salta si es cero
90      GOTO     BucleInicio
91
92      BSF      LATC, 0          ; RCO-Led_piloto ENCENDIDO
93      ;-----
94      Reiniciar_Todo:
95      CLRF      LATD          ; Apaga todos los LEDs del Puerto D
96      CLRF      Flag          ; Limpia
97      MOVLW     d'30'          ;
98      MOVWF     Contador15s    ; 30 interrupciones para los 15s
99      ;-----
100     EncuestaON:              ; Espera a S2 para iniciar secuencia
101     BTFSC     PORTC, 4
102     GOTO      EncuestaON
103     CALL      Delay100ms
104     Est_transistorio:          ; Esperar a que suelte S2
105     BTFSS     PORTC, 4
106     GOTO      Est_transistorio
107
108     ;***** SECUENCIA OLA ***** ; Inicio de la secuencia
109     solicitada
110     MOVLW     b'00000011'      ; LED izq ON (rojos) siempre encendido
111     MOVWF     LATD
112     BSF      Flag, 0          ; 0: Interr. empieza a contar
113
114     Ola_Hacia_Izq:

```

```

114      BTFSC    Flag, 1
115      GOTO     Reiniciar_Todo
116
117      MOVLW    d'100'
118      MOVWF    Xveces
119      Espera_Izq:
120
121      CALL     Delay1ms          ; Espera 1ms
122      BTFSS    PORTC, 5          ; Revisa S3 cada milisegundo
123      GOTO     Detener_Secuencia
124      DECFSZ   Xveces, 1
125      GOTO     Espera_Izq
126
127      BCF      Sentido, 0        ; bandera indicar sent. izq.
128
129      RLNCF    LATD, 1           ; Rota la luz a la izquierda
130      BTFSS    LATD, 7           ; 10000000 (bit 7 en 1)
131      GOTO     Ola_Hacia_Izq    ;
132
133      Ola_Hacia_Der:
134      BTFSC    Flag, 1
135      GOTO     Reiniciar_Todo
136
137      MOVLW    d'100'
138      MOVWF    Xveces
139      Espera_Der:
140      CALL     Delay1ms          ; Espera 1ms
141      BTFSS    PORTC, 5          ; Revisa S3 cada milisegundo
142      GOTO     Detener_Secuencia
143      DECFSZ   Xveces, 1
144      GOTO     Espera_Der
145
146      BSF      Sentido, 0        ; bandera indicar sent. DER.
147
148      RRNCF    LATD, 1           ; Rota la luz a la derecha
149      BTFSS    LATD, 0           ; 00000001 (bit 0 es 1)
150      GOTO     Ola_Hacia_Der
151
152      GOTO     Ola_Hacia_Izq    ; Repite la Ola
153
154 ; ----- DETENER Y NO OFF
155      Detener_Secuencia:
156      CALL     Delay100ms        ; Anti-rebote
157      Boton_S3_D:
158      BTFSC    Flag, 1           ; Tiempo pasado incluso en pausa
159      GOTO     Reiniciar_Todo
160      BTFSS    PORTC, 5          ; Espera que se suelte S3
161      GOTO     Boton_S3_D
162
163
164 ; ----- CONTINUA S2
165      Trans_s2:
166      BTFSC    Flag, 1
167      GOTO     Reiniciar_Todo
168      BTFSC    PORTC, 4
169      GOTO     Trans_s2
170      CALL     Delay100ms        ; Anti-rebote
171      Continuar_s2:
172      BTFSC    Flag, 1
173      GOTO     Reiniciar_Todo
174      BTFSS    PORTC, 4
175      GOTO     Continuar_s2
176

```

```

177      BTFSC    Sentido, 0      ; Esta en dir. derecha
178      GOTO     Ola_Hacia_Der   ; va a derecha
179      GOTO     Ola_Hacia_Izq   ; No, va a izquierda
180
181      ;++++++ DELAY DE X[ms] ++++++
182      ;Deberia llamar a Delay1ms, Xveces
183      DelayXms:
184          MOVWF  Xveces
185      bucleVeces:
186          CALL   Delay1ms
187          DECFSZ  Xveces,1
188          GOTO    bucleVeces
189
190          return
191      ;+++++
192
193      ;++++++ DELAY DE 100MS ++++++
194      Delay100ms:
195          CALL   Delay1ms
196          DECFSZ  ContaDelay100,1
197          GOTO    Delay100ms
198          ;Reinicio ContaDelay100 = 100
199          MOVLW   d'100' ;W=100
200          MOVWF   ContaDelay100
201
202          return
203      ;+++++
204      ;++++++ DELAY DE 1MS ++++++
205      Delay1ms:
206          DECFSZ  Contador2,1
207          GOTO    bucle1
208          GOTO    salir
209      bucle1:
210          DECFSZ  Contador1,1
211          GOTO    bucle1
212
213          MOVLW   d'198' ;W = 198
214          MOVWF   Contador1 ;Contador1 = 198
215          GOTO    Delay1ms
216      salir:
217          MOVLW   d'21' ;W = 21
218          MOVWF   Contador2 ;Contador2 = 10
219          return
220      ;+++++
221
222      ; *****INTERRUPCION_HP*****
223      InterrupcionHP:
224          ; Complementa LED RCO (D9)
225          BTG     LATC, 0      ; Toggle LED piloto 1 Hz
226          BCF     INTCON, TMROIF ; OFF bandera IT TIMER
227
228          MOVLW   0xD9      ; 65535 (55769) ---> CAMBIA HEXADECIMAL
229          MOVWF   TMROH      ; PARTRE ALTA 16 BITS
230          MOVLW   0xDA      ; AL CAMBIAR CF A D5 ESTE CUENTA MENOS Y ASI
231          MOVWF   TMROL      ; PARTRE BAJA 16 BITS
232
233          ; ----- 15 seg:
234          BTFSS   Flag, 0      ; Esta On en conteo
235          GOTO    Fin_ISR      ; No hay que contar: sale programa
236
237          DECFSZ  Contador15s, 1 ; S : decrementa Contador.
238          GOTO    Fin_ISR      ; No lleg a 0: sale
239

```

```

240      ;                               Cont. en 0 = pasaron 15 segundos exactos
241      BCF      Flag, 0                ; 0: detiene el conteo
242      BSF      Flag, 1                ; Continúa contando
243
244      Fin_ISR:
245      RETFIE                          ; Regresa de interrupción
246      ;+++++++
247      end
248      ;+++++++

```

Listing 6: Código Parte 3 - Secuencia Ola con Temporizador de 15 Segundos y pausas

### 3.6.3. Lógica de Temporización

Se utiliza un contador que se decrementa en cada interrupción (cada 500 ms):

$$Interrupciones_{necesarias} = \frac{15 \text{ s}}{0,5 \text{ s}} = 30 \text{ interrupciones}$$

Variables utilizadas:

```

1 Contador15s      equ 0x06          ; Cuenta 30 interrupciones (15 seg)
2 Flag             equ 0x07          ; 0=Conteo ON, 1=OFF(Ya está los 15 seg)

```

Inicialización:

```

1 Reiniciar_Todo:
2   CLRF    LATD                ; Apaga LEDs
3   CLRF    Flag                ; Limpia Bandera
4   MOVLW   d'30'
5   MOVWF   Contador15s         ; 30 x 500ms = 15s

```

### 3.6.4. Modificaciones en la ISR

```

1      InterrupcionHP:
2      ; Complementa LED RCO (D9)
3      BTG    LATC, 0           ; Toggle LED piloto 1 Hz
4      BCF    INTCON, TMR0IF     ; OFF bandera IT TIMER
5
6      MOVLW  0xD9               ; 65535 (55769) ---> CAMBIA HEXADECIMAL
7      MOVWF  TMR0H               ; PARTE ALTA 16 BITS
8      MOVLW  0xDA               ; AL CAMBIAR CF A D5 ESTE CUENTA MENOS Y ASI
9      MOVWF  TMR0L               ; PARTE BAJA 16 BITS
10
11     ; ----- 15 seg:
12     BTFSS   Flag, 0            ; Esta On en conteo
13     GOTO    Fin_ISR           ; No hay que contar: sale programa
14
15     DECFSZ   Contador15s, 1     ; S : decrementa Contador.
16     GOTO    Fin_ISR           ; No llegó a 0: sale
17
18     ;                               Cont. en 0 = pasaron 15 segundos exactos
19     BCF      Flag, 0            ; 0: detiene el conteo
20     BSF      Flag, 1            ; Continúa contando
21
22     Fin_ISR:
23     RETFIE                      ; Regresa de interrupción

```

Listing 7: ISR con contador de 15 segundos

### 3.6.5. Control en el Programa Principal

Antes de cada transición de la secuencia, se verifica si el tiempo se cumplió:

```
1 Ola_Hacia_Izq:  
2   BTFSC   Flag, 1           ; Tiempo cumplido?  
3   GOTO    Reiniciar_Todo    ; Si: reinicia todo  
4  
5   ; Código ...
```

### 3.6.6. Activación del Contador

El contador se activa cuando se presiona S2 por primera vez:

```
1 ; Despues de presionar S2:  
2 BSF      Flag, 0           ; Activa el conteo de 15s
```

#### Comportamiento esperado:

- Si se presiona S2 y no se toca nada más, la secuencia funciona exactamente 15 segundos.
- Si se pausa con S3, el contador sigue corriendo en segundo plano.
- Al cumplirse los 15 segundos (incluso durante una pausa), todo se apaga y regresa a condiciones iniciales.
- El LED piloto continúa parpadeando a 1 Hz durante todo el proceso.

### 3.7. PARTE 4: Temporizador de 140 Segundos (Adional)

Aunque esta parte no se solicita en el enunciado de la practica se incluye para ver como se configuraria para un mayor tiempo. Esta parte extiende la funcionalidad a 140 segundos (2 minutos y 20 segundos). La lógica es similar a la Parte 3, pero requiere un contador adicional.

#### 3.7.1. Problema de Rango

Con interrupciones cada 500 ms:

$$\text{Interrupciones} = \frac{140 \text{ s}}{0,5 \text{ s}} = 280 \text{ interrupciones}$$

Como 280 excede el rango de un byte (0-255), se implementa un contador de dos niveles:

```
1 Contador15s      equ 0x06      ; Contador de segundos
2 ContadorMedioSeg equ 0x08      ; Contador de medios segundos
```

Inicialización:

```
1 MOVLW    d'140'
2 MOVWF    Contador15s          ; 140 segundos
3 MOVLW    d'2'
4 MOVWF    ContadorMedioSeg     ; 2 x 500ms = 1 segundo
```

#### 3.7.2. ISR Modificada

```
1 InterrupcionHP:
2   BTG     LATC, 0
3   BCF     INTCON, TMR0IF
4
5   MOVLW   0xD9
6   MOVWF   TMR0H
7   MOVLW   0xDA
8   MOVWF   TMR0L
9
10  ; ----- 140 seg:
11  BTFSS    Flag, 0
12  GOTO     Fin_ISR
13
14  DECFSZ   ContadorMedioSeg, 1
15  GOTO     Fin_ISR          ; Aun no pasa 1 segundo
16
17  ; 2 interrupciones = 1 SEGUNDO
18  MOVLW   d'2'
19  MOVWF    ContadorMedioSeg
20
21  DECFSZ   Contador15s, 1
22  GOTO     Fin_ISR
23
24  ; Pasaron 140 segundos
25  BCF      Flag, 0
26  BSF      Flag, 1
27
28 Fin_ISR:
29  RETFIE
```

Listing 8: ISR para 140 segundos

Explicación:

1. Cada interrupción decrementa ContadorMedioSeg.



2. Cuando `ContadorMedioSeg` llega a 0 (cada 1 segundo), se recarga y se decrementa `Contador15s`.
3. Cuando `Contador15s` llega a 0 (después de 140 segundos), se activa la bandera de fin.

## 4. Pruebas y Verificaciones

### 4.1. Parte 1: Secuencia Ola con Pausa

#### Prueba 1: Inicio de secuencia

- Se presionó RESET (S1)
- El LED piloto parpadeó 20 veces correctamente
- Se presionó S2
- La secuencia ola inició correctamente con 2 LEDs base encendidos

#### Prueba 2: Pausa y continuación

- Durante la secuencia se presionó S3
- Los LEDs se mantuvieron en su estado (no se apagaron)
- Al presionar S2 nuevamente, la secuencia continuó desde el mismo punto
- El movimiento mantuvo la dirección correcta (izquierda o derecha)

### 4.2. Parte 2: Timer0 con LED Piloto

#### Prueba 1: Frecuencia de parpadeo

- Se midió el período del LED piloto con un cronómetro
- Resultado medido: aproximadamente 1 segundo por ciclo completo (ON - OFF)

#### Prueba 2: Independencia de la secuencia

- El LED piloto continuó parpadeando durante la ejecución de la secuencia
- El LED piloto continuó parpadeando durante las pausas
- No se observó interferencia entre ambos procesos

### 4.3. Parte 3: Temporizador de 15 Segundos

#### Prueba 1: Duración sin pausas

- Se inició la secuencia con S2
- Se cronometró el tiempo hasta el apagado automático
- Resultado:  $\approx 15$  segundos.

#### Prueba 2: Duración con pausas

- Se inició la secuencia con S2
- A los 5 segundos se pausó con S3
- Se esperó 3 segundos en pausa

- Se continuó con S2
- Tiempo total hasta apagado:  $\approx 15$  segundos desde el primer S2

### **Prueba 3: Apagado durante pausa**

- Se inició la secuencia
- A los 10 segundos se pausó
- Sin presionar S2, todo se apagó a los 15 segundos

## **5. Códigos Completos y Videos de Funcionamiento**

Los códigos completos implementados en lenguaje ensamblador para las partes de la práctica, así como los videos que demuestran su funcionamiento, están disponibles en el repositorio de GitHub:

### **Códigos fuente (.asm):**

- [Repositorio completo - Práctica 2 \(4 códigos\)](#)

### **Videos de demostración (.mp4):**

- [Video Parte 1 - Secuencia con Pausa](#)
- [Video Parte 2 - Timer0 y LED Piloto](#)
- [Video Parte 3 - Temporizador 15 segundos](#)

### **Repositorio principal:**

- [Repositorio Principal - Todas las Prácticas](#)

## 6. Preguntas

1. ¿Cuáles son los registros y bits asociados al funcionamiento del Timer que seleccionó para la práctica?

Para el Timer0 del PIC18F4550, los registros principales son:

- **T0CON (Timer0 Control Register):** Registro de configuración principal
  - TMR0ON (bit 7): Habilita/deshabilita el Timer0
  - T08BIT (bit 6): Selecciona modo 8 o 16 bits
  - T0CS (bit 5): Selecciona fuente de reloj
  - T0SE (bit 4): Selecciona flanco (solo modo contador)
  - PSA (bit 3): Asigna prescaler
  - T0PS2:T0PS0 (bits 2-0): Configuran valor del prescaler
- **TMR0H y TMR0L:** Registros que contienen el valor actual del contador (modo 16 bits)
- **INTCON:** Registro de control de interrupciones
  - GIE (bit 7): Habilita interrupciones globales
  - TMR0IE (bit 5): Habilita interrupción del Timer0
  - TMR0IF (bit 2): Bandera de desbordamiento del Timer0

*Los bits utilizados se aprecian en la Fig. 3 y 4*

2. ¿Para qué sirve la opción de **PRE-SCALER** del módulo temporizador del microcontrolador?

El prescaler es un divisor de frecuencia que permite reducir la velocidad a la que se incrementa el contador del Timer0. Su función principal es *permitir medir tiempos más largos* sin necesidad de recargar constantemente el timer desde software.

Por ejemplo, si trabajamos con un reloj de 5 MHz ( $T_{cy} = 0.2 \mu s$ ):

- Sin prescaler: tiempo máximo =  $65536 \times 0.2 \mu s = 13.1 ms$
- Con prescaler 1:256: tiempo máximo =  $65536 \times 0.2 \mu s \times 256 = 3.35 s$

Esto permite generar interrupciones cada 500 ms como se requiere en la práctica, algo imposible sin prescaler.

3. ¿Qué es una interrupción?

Una interrupción es un mecanismo que permite al microcontrolador *suspender* temporalmente la ejecución del programa principal para *atender un evento* de mayor prioridad. Cuando ocurre una interrupción:

- a) El microcontrolador guarda el estado actual (contexto)
- b) Salta a una dirección de memoria específica (vector de interrupción)
- c) Ejecuta una rutina de servicio de interrupción (ISR)
- d) Restaura el contexto y retorna al programa principal

Las interrupciones son útiles para:

- Atender eventos en tiempo real
- Mejorar la eficiencia del programa (no hace falta estar verificando constantemente)

- Realizar múltiples tareas aparentemente simultáneas

#### 4. ¿Qué función desempeña el vector de interrupción?

El vector de interrupción es una dirección de memoria específica donde el microcontrolador busca el código a ejecutar cuando ocurre una interrupción. En el PIC18F4550 existen dos vectores:

- **Vector de Alta Prioridad (0x0008):** Para interrupciones urgentes
- **Vector de Baja Prioridad (0x0018):** Para interrupciones menos críticas

En la práctica, colocamos un salto en el vector de alta prioridad:

```

1  org      0x08
2  goto     InterruccionHP
3

```

Dirige el programa a nuestra rutina de servicio cuando el TMR0 genera una interrupción.

#### 5. ¿Para qué se activa el bit TMR0IF una vez que ha sucedido el desborde del módulo temporizador TMR0?

El bit TMR0IF (Timer0 Interrupt Flag) se activa automáticamente cuando el Timer0 se desborda (pasa de 0xFFFF a 0x0000 en modo 16 bits). Este bit cumple dos funciones importantes:

- a) **Indicador de evento:** Permite saber que ocurrió un desbordamiento, incluso si las interrupciones están deshabilitadas.
- b) **Generador de interrupción:** Si TMR0IE está habilitado, cuando TMR0IF se activa se genera una interrupción que hace que el programa salte al vector de interrupción.

Es crucial que la rutina de servicio de interrupción limpie manualmente este bit con BCF INTCON, TMR0IF, ya que si no se limpia, la interrupción se volverá a generar inmediatamente al salir de la ISR, creando un bucle infinito.

#### 6. ¿Por qué es conveniente un módulo temporizador operando a 16 bits para conteos de tiempo relativamente grandes?

Un temporizador de 16 bits puede contar hasta 65536 valores, mientras que uno de 8 bits solo hasta 256. Esto significa:

**Ventajas del modo 16 bits:**

- **Mayor rango de tiempo:** Con prescaler 1:256, se pueden medir hasta 3.35 segundos en una sola cuenta.
- **Menos interrupciones:** Para medir 15 segundos necesitamos solo 30 interrupciones con timer de 16 bits. Con uno de 8 bits necesitaríamos muchas más, sobrecargando el procesador.
- **Mayor precisión:** Al tener menos interrupciones, hay menos acumulación de errores por latencia de atención.
- **Código más simple:** No se necesitan contadores adicionales en software para alcanzar tiempos largos.

**Comparación práctica:**

- 8 bits + prescaler 256: máximo 13.1 ms por interrupción

- 16 bits + prescaler 256: máximo 3.35 s por interrupción

Para nuestra aplicación de 500 ms, el modo 16 bits es ideal.

**7. Muestre los cálculos realizados y los valores que cargó en los registros correspondientes del TMR para lograr la interrupción cada 500ms.**

**Datos iniciales:**

- Frecuencia del cristal:  $F_{osc} = 20 \text{ MHz}$
- Frecuencia de instrucción:  $F_{cy} = \frac{F_{osc}}{4} = \frac{20 \text{ MHz}}{4} = 5 \text{ MHz}$
- Período de instrucción:  $T_{cy} = \frac{1}{F_{cy}} = \frac{1}{5 \text{ MHz}} = 0,2\mu s$
- Prescaler seleccionado: 1:256
- Tiempo deseado:  $T_{deseado} = 500 \text{ ms}$

**Paso 1: Calcular período con prescaler**

$$T_{tick} = T_{cy} \times Prescaler = 0,2\mu s \times 256 = 51,2\mu s$$

**Paso 2: Calcular número de cuentas necesarias**

$$Cuentas = \frac{T_{deseado}}{T_{tick}} = \frac{500 \times 10^{-3} s}{51,2 \times 10^{-6} s} = \frac{0,5 s}{51,2\mu s} = 9765,625$$

Redondeamos a 9766 cuentas.

**Paso 3: Calcular valor de precarga**

En modo 16 bits, el timer cuenta desde el valor de precarga hasta 65535 (0xFFFF), luego se desborda a 0. Para que cuente 9766 veces:

$$Valor_{precarga} = 65536 - 9766 = 55770_{10}$$

**Paso 4: Convertir a hexadecimal**

$$55770_{10} = D9DA_{16}$$

Por lo tanto:

- TMR0H = 0xD9 (parte alta)
- TMR0L = 0xDA (parte baja)

**Verificación:**

$$T_{real} = 9766 \times 51,2\mu s = 500,0192ms$$

$$Error = \frac{|500,0192 - 500|}{500} \times 100 \% = 0,0038 \%$$

El error es despreciable para esta aplicación.

La solución más detallada y las conversiones se encuentran en la sección de resolución desde la parte 2 y ver la Fig. 9.

**Código de implementación:**

```

1 ; Configuración del Timer0
2 BCF      TOCON, TMROON    ; Apaga Timer0
3 BCF      TOCON, T08BIT    ; Modo 16 bits
4 BCF      TOCON, TOCS      ; Reloj interno
5 BCF      TOCON, PSA       ; Prescaler habilitado
6 BSF      TOCON, TOPS2     ; \
7 BSF      TOCON, TOPS1     ; > Prescaler = 256 (111 binario)
8 BSF      TOCON, TOPS0     ; /
9
10 ; Valor para 500 [ms]
11 MOVLW    0xD9
12 MOVWF    TMROH           ; Parte alta = D9
13 MOVLW    0xDA
14 MOVWF    TMROL           ; Parte baja = DA
15
16 BSF      TOCON, TMROON    ; Enciende Timer0
17

```

## 7. Conclusiones y Recomendaciones

### 7.1. Conclusiones

Con base a la resolución de esta práctica se concluye que:

- Se logró configurar exitosamente el módulo Timer0 del PIC18F4550 en modo 16 bits con prescaler de 256, para obtener 500 ms.
- La implementación de interrupciones permite ejecutar tareas periódicas (parpadeo del LED piloto) de forma independiente del programa principal, mejorando la eficiencia del código.
- El uso del prescaler demostró ser fundamental para alcanzar temporizaciones largas (500 ms) que serían imposibles de lograr sin este divisor de frecuencia.
- La rutina de servicio de interrupción (ISR) debe ser breve y eficiente para no interrumpir excesivamente el programa principal.
- El sistema de banderas (Flag) permitió coordinar eficientemente el conteo de tiempo entre la ISR y el programa principal sin necesidad de variables globales complejas.
- El contador de 15 segundos implementado en la ISR demostró que es posible controlar tiempos largos usando interrupciones cortas, simplemente contando el número de veces que ocurre la interrupción.
- La independencia entre el LED piloto (controlado por interrupciones) y la secuencia principal (controlada por bucles) demuestra que el PIC18F4550 puede manejar múltiples tareas aparentemente simultáneas.
- Es fundamental limpiar la bandera TMR0IF dentro de la ISR para evitar re-entradas infinitas de la interrupción, lo cual causaría que el programa se cuelgue.

### 7.2. Recomendaciones

- Siempre verificar con un osciloscopio o cronómetro la precisión de las temporizaciones implementadas, especialmente en aplicaciones críticas donde el tiempo es importante.
- Usar el modo de 16 bits siempre que sea posible para temporizaciones largas, ya que reduce significativamente el número de interrupciones y mejora la precisión.

## Referencias

- [1] Microchip Technology Inc., *PIC18F2455/2550/4455/4550 Data Sheet: High Performance, Enhanced Flash, USB Microcontrollers with nanoWatt Technology*, DS39632E, 2024. [En línea]. Disponible en: <https://ww1.microchip.com/downloads/en/devicedoc/39632e.pdf>
- [2] Microchip Technology Inc., *PICkit 3 Programmer/Debugger User's Guide*, DS51795B, 2024. [En línea]. Disponible en: <https://ww1.microchip.com/downloads/en/DeviceDoc/51795B.pdf>
- [3] F. E. Valdes-Perez y R. Pallas-Areny, *Microcontrollers: Fundamentals and Applications with PIC*, CRC Press, 2009. [En línea]. Disponible en: <https://www.embedic.com/uploads/files/20201008/Microcontrollers%20-%20Fundamentals%20and%20Applications%20with%20PIC.pdf>